FRACTL · STAGES 2–3 · MEMBER 3

Tooling, Verification & FRACTL Studio

How the project is built, proven and shown: the live studio with its 2D/3D views and GPU renderer, the tests that prove nothing old broke, and the offline bundle for the presentation laptop.

Flow: editor → studio.py (local server) → real `fractlc --jit` → .svg / .f3d → browser: Canvas 2D or **WebGL2 on the GPU** → the model grows in drawing order

IN ONE BREATH

In Stage 1, Member 3's tools showed results the compiler had produced in advance. Now **FRACTL Studio** compiles whatever you type, live, with the real LLVM compiler, and grows the result on screen in the exact order the compiled program drew it — in 2D or in 3D on the GPU, with shadows and wind. Every Stage 1 tool is still there, and four test suites prove that all build modes draw identical pictures.

01 What changed since Stage 1

Area	Stage 1	Now
Front end	web form → SVG; static stage viewer	Stage 3 **FRACTL Studio**: editor, live compiler pipeline with timings, 2D/3D toggle, growth replay, Stages tab, Build log, Form, IR view, export — the old form and viewer are kept too
Rendering	browser shows an SVG file	2D replay with a glowing "turtle"; **3D WebGL2 renderer** written from scratch (no libraries)
Tests	frontend suite, 1,904 checks	Stage 2 runtime, pipeline and golden-file suites · Stage 3 mode-equivalence suite; frontend now 79,580 checks
Build	`make`, `make test`	generic demo targets, `make demo / golden / test / bundle`
Delivery	run from the source folder, LLVM installed	**offline bundle**: one folder / zip, static binaries, launcher, setup check

02 Stage 2 tooling Stage 2

- **New test suites**: `tests/test_runtime.c` (calls the runtime directly and checks exact SVG coordinates), `tests/test_pipeline.sh` (full `.fr → fractlc → link → run`, bracket errors, and **Stage 1 regression**: triangle and Koch byte-identical to Stage 1), `tests/run_golden.sh` (every demo's output and IR compared to saved "golden" copies).
- **Viewer bug found and fixed**: Windows PowerShell's JSON reader treats keys case-insensitively, so `"F"` and `"f"` (pen-up) collided and the viewer build crashed on the carpet. Switched to a case-sensitive parser.
- **Web form**: five new presets (plant, tree, bush, weed, carpet) and updated symbol help.

03 FRACTL Studio — the server Stage 3

`studio.py` uses only Python's standard library. Each request writes the editor's text to a real `.fr` file and runs the real tools:

Endpoint	What it runs
`POST /api/compile`	`fractlc input.fr --jit --out … --emit-ir … --timings` (or object + `cc` + run). Returns the drawing, the IR, each phase's time, and every command's output.
`POST /api/inspect`	`fr_frontend --json` (optional `--fractal`, `--iterations`) — the live Stages tab
`POST /api/ir`	`fractlc --ir-only` in recursive, unrolled or `--opt` mode — IR side by side
`GET /api/presets`	the 17 example grammars

Three real-world problems solved along the way:

- **Slow disk**: from WSL, reading and writing Windows folders is slow — writing the meadow's output took about 0.3 s instead of 0.04 s. The server copies the compiler into Linux `/tmp` once and builds there.
- **"localhost" delay**: Windows tries IPv6 first for `localhost`, but WSL forwards IPv4 and IPv6 separately, so every request waited 2 s for a fallback. The server listens on **both**; every address now answers in about 4 ms.
- **Two builds at once** would overwrite each other's files, so builds are serialized with a lock.

04 FRACTL Studio — the page Stage 3

Plain HTML, CSS and JavaScript; no framework, no CDN, no web fonts — it works with the network cable unplugged.

File	Job
`app.js`	Editor with syntax highlighting and error-line marking; Build; the **compiler pipeline** panel (each phase lights up with its measured time); the growth timeline (play, scrub, speed); the **2D/3D toggle** (each mode keeps its own source and last build); LLVM IR drawer; export to `.svg` / `.obj`
`render2d.js`	Replays the SVG stroke by stroke in drawing order; colour runs teal → amber with the drawing order; finished strokes are cached so big curves stay smooth
`render3d.js`	The 3D renderer (next section)
`stages.js`	Stage 1 features, live: **Stages** tab, **Build log** tab, **Form** editor

05 The 3D renderer — how the model "grows" Stage 3

Written directly against **WebGL2** (no three.js). The ideas, in order:

1. **Instancing**: there is only one small cylinder mesh and one small blade mesh. Each of the ~40,000 pieces is an *instance* — its start, end, radius, colour — so the GPU builds all the geometry in one draw call per kind.
2. **Birth times**: each piece gets a time from its position in the `.f3d` stream. In the vertex shader a piece is invisible before its birth, then **grows out of its base** over a fraction of a second, glowing while it's new. That is the "watch it being built" effect — it replays the exact order the compiled program drew. *Together* mode lets every instance grow at once (grass sprouts while the tree climbs); *Emission order* is strictly sequential.
3. **Grass**: consecutive blade pieces are joined into whole blades; each piece knows how far along its blade it sits. That gives a pointed tip, dark-root-to-light-tip shading, occasional dry tips, and **wind** that bends each blade by the square of that distance (roots stay put).
4. **Light**: sky + sun lighting; blades let light through when the sun is behind them; distance fog; the newest piece is marked by a glowing light.
5. **Shadows**: a **shadow map** — the scene is first drawn from the sun into a 2048×2048 depth texture; then each pixel checks whether it is hidden from the sun (9 filtered samples for soft edges). The tree shades the grass

and the grass shades the ground.

6. **Bark**: procedural, in each branch's own coordinates so it wraps without a seam: long fissures, dark cracks, fine grain, moss near the ground.

> **SAY IT LIKE THIS**
> "The browser never invents geometry. It receives the exact list of pieces the compiled program wrote, in the order it wrote them, and the GPU grows each one at its moment. On this laptop's integrated Intel graphics that's 40,000 pieces with shadows and wind at 60 frames per second."

06 Stage 1 features — kept, and now live

Stage 1 feature	Where it is now
Stage viewer (source, tokens, AST, expansion, turtle IR, counts, iteration slider, IR before/after)	**Stages tab**, computed live for whatever is in the editor; IR recursive / unrolled / `--opt` / side by side. The static `viewer/index.html` is also kept and linked.
Web form (axiom, rules, angle, …)	**Form** switch in the editor; original `web_ui.py` kept as "Launch FRACTL Classic UI.bat" (port 8733)
Build log + generated source	**Build log** tab: every command, its output and time
Terminal workflow and scripts	unchanged (`fractl-run.sh`, `fractl-lsystem.sh`, `make koch` …); the Stages tab lists the exact commands

07 Verification — proving nothing broke

Suite	Result	What it proves
`tests/test_frontend.cpp`	79,580 / 0 failed	lexer, parser, brackets, random rules (incl. the distribution of choices), 3D symbols, scenes, placement, every error message, every demo's command counts
`tests/test_runtime.c`	36 / 0 failed	exact 2D and 3D coordinates, push/pop, instance rotation and scale, random numbers repeatable, underflow error
`tests/test_pipeline.sh`	8 / 0 failed	end-to-end builds, compile-time errors, Stage 1 pictures byte-identical
`tests/test_modes.sh`	59 / 0 failed	for all 17 demos: **recursive = unrolled**, **object+link = JIT**, **--opt changes nothing**, Stage 2 pictures identical; recursive IR far smaller
`tests/run_golden.sh`	16 / 16 files	outputs and IR match saved reference copies
viewer smoke test	40 / 0 failed	the static stage viewer renders and degrades correctly

One command, `make test`, runs everything from a clean build — with zero compiler warnings.

> **THE THROUGHLINE (UNCHANGED SINCE STAGE 1)**
> No tool fakes a result: the studio runs the real compiler on every Build, the tests compare exact bytes, and every new feature had to leave every old picture unchanged.

08 The offline bundle

`make bundle` produces `dist/FRACTL-Studio` (and a 15 MB zip): static `fractlc` and `fr_frontend`, the runtime, the studio, the demos, the classic UI, the stage viewer and the docs. The presentation laptop only needs WSL with Python.

- **Launch FRACTL Studio.bat**: starts the server in WSL, waits until it answers, and opens Edge as a clean app window.
- **Check FRACTL Setup.bat**: checks WSL, Python and the compiler, and does a real JIT build of the meadow with its time.
- A settings switch turns shadows and wind off for a very weak GPU (not needed on this laptop: 60 fps on integrated graphics).

09 Numbers to quote

~0.14 s edit → build → on screen (meadow, round trip)	**60 fps** 39,588 pieces, shadows + wind, integrated Intel GPU	**0** internet, CDN or library dependencies
79,580 frontend checks (Stage 1: 1,904)	**59** build-mode equivalence checks	**17** example grammars, 2D and 3D

10 Questions you may be asked

Is the growth animation pre-rendered?

No. Each Build runs the compiler; the animation replays the pieces the compiled program just wrote, in its drawing order. Change the seed and you get a different tree.

Why write your own renderer instead of using three.js?

No third-party code, nothing to download, and full control: growth from birth times, grass shading and wind all live in our own shaders.

What does the GPU do, and what does the CPU do?

The CPU compiles and runs the program (well under a second). The GPU draws: instancing, growth, lighting, shadows and wind.

How do you know the new compiler draws the same as the old one?

The mode-equivalence and golden tests compare output files byte for byte across every build mode and against the Stage 1 and Stage 2 results.

What if the presentation laptop has no WSL?

The setup checker says so and explains the one-time install; the compiler itself needs nothing else because it is fully static.

FRACTL Stages 2–3 · tooling, verification and studio (Member 3) · studio.py, studio/ (app.js, render2d.js, render3d.js, stages.js), Makefile, build.ps1, tests/, viewer/, web_ui.py, tools/